Senior

As a Senior Data Scientist, you should expand your portfolio of skills to add the following:

General AI Knowledge

Fields of AI

- Machine Learning
  - Simulation
  - Discrete Event Simulation
  - Agent Based Modelling
  - System Dynamics / Numerical Partial Derivative Systems
  - Optimization
  -

Computer Vision / Signal Processing

Natural Language Processing

Generative AI

Deep Learning

Layers in a neural network (Fully Connected, Convolutional, Recurrent, Dropout, Pooling, Batch, Normalization, Attention)

Learning schemas (SGD, Adam, adaptive models, batch size/ learning rate tradeoff etc)

Loss functions (L-based distance, cross-entropy, others)

- Evaluation
  - Bias / Variance tradeoff

Epoch graph vs Dataset Size graphs

- Error analysis
  - What is it? What options do you have?
  - What are the options when an NN does not give good results? Why?

Variable normalization / feature scaling

Is linear separability a requirement before using logistic regression? Or is it desirable?

SVMs: pros and cons

NN

ReLU

How do you avoid the infamous racial bias on object detection (detecting a family of African-Americans as apes)

What do algorithms like Word2Vec (or ULMFit, or encoder LLMs) do? How does it work?

- Define "embeddings"
  - Why "negative sampling"?

Pros cons when modeling sequence

- LSTM
  - 1d convolution

Explain differences, and how the approaches differ, e.g. with "objects"

Detection
Semantic segments
Instance segments

Vector storage Indexing, search

NLP
- Explain TDF-IDF, LDA, Bag of Words.
LLMs / GenAl / Agents
- MoE (Mixture of Experts): Understanding how models like GPT-4 or Mixtral activate only specific "expert" sub-networks to save on inference costs.
Fine-Tuning (PEFT/LoRA): Knowing the math of "Low-Rank Adaptation"-how to update a tiny fraction of weights to specialize a model without retraining the whole thing.
-Knowledge Graphs: Moving beyond Vector DBs to graph-based relationships for comillex
RAG.
KV Caching: Understanding how the model "remembers" previous tokens in a session to save compute power and reduce latency.
Guardrails & Safety: Implementing NeMo or LlamaGuard (Understanding the "Alignment" problem).
LLM-as-a-Judge (G-Eval): Building a deterministic rubric for an LLM to grade another LLM.
Agentic Frameworks & Patterns: Mastery of orchestration patterns like Evaluator-Optimizer, Routing, and Parallelization.

- Advanced RAG: Deep knowledge of ReAG (Reasoning Augmented Generation) and how to improve retrieval using Knowledge Graphs.
Transcript Evaluation: Analyzing the path the agent took (did it waste tokens? did it get
stuck in a loop?).
Outcome Evaluation: Measuring if the agent actually solved the business problem (e.g.,
"Was the flight actually booked?").
  - Trajectory Success Rate: If an agent uses 5 tools to solve a task, did it take the most efficient path (Shortest Path) or did it "wander" through unnecessary tool calls?
    Agentic Al vs. Al Agents: Can distinguish between building a single autonomous script (Agent) and building a system where Al is embedded into the whole business process

Advanced RAG: Deep knowledge of ReAG (Reasoning Augmented Generation) and how to

improve retrieval using Knowledge Graphs.
Transcript Evaluation: Analyzing the path the agent took (did it waste tokens? did it get stuck in a loop?).
Outcome Evaluation: Measuring if the agent actually solved the business problem (e.g),
"Was the flight actually booked?").
Trajectory Success Rate: If an agent uses 5 tools to solve a task, did it take the most efficient path (Shortest Path) or did it "wander" through unnecessary tool calls?
Agentic Al vs. Al Agents: Can distinguish between building a single autonomous script (Agent) and building a system where Al is embedded into the whole business process
(Agentic Al).
-Agent frameworks: LangChain/LangGraph, AutoGen, CrewAl, among others.
Optimization

- Solving speed on a MIP (Mixed-Integer Programming) solver

If you want to speed up solving time, what do you change? Objective function,

- variables, constraints?
- What is the trade-off of heuristics on optimization models?

Development

- How would you handle an Agile methodology for doing data science?
  - Why not just ask people instead of modelling preferences to predict behavior?
  - Be able to state explicit hypotheses to validate through data (How does the world work?)
  - What don't I know about the world? -> Discover through data
  - What are the uncharted territories? -> Use data as an exploration
  - The art of breaking up a project into User Stories
  - How to generate a wealth of hypothesis and interpretations from a dataset /
  - visualizations / a problem statement / a business context

OLS: https://web.stanford.edu/~mrosenfe/soc_meth_proj3/matrix_OLS_NYU_notes.pdf

Discrete Event Simulation (DES) is a modeling technique where a system's operation is represented as a chronological sequence of distinct events. Unlike continuous simulations, where state variables change smoothly over time, DES assumes the system remains unchanged between consecutive events, allowing the simulation clock to "jump" directly to the next scheduled even

Interview Cheat Sheet

CONCEPT	ONE-LINE DEFINITION	DS APPLICATION
Partial Derivative	Rate of change w.r.t. one variable, others fixed	Feature sensitivity, gradients
Gradient	Vector of all partial derivatives	Gradient descent, backprop
Numerical Diff	Approximate derivative by finite step h	Gradient checking, debugging
Chain Rule	∂L/∂w = product of local gradients through chain	Backpropagation in NNs
ODE / System Dynamics	dx/dt = f(x,t) — state evolves via derivatives	Epidemic models, churn, forecasting
Jacobian	Matrix of all 1st partials for vector functions	Auto-diff, sensitivity analysis
Hessian	Matrix of 2nd partials — curvature info	Newton's method, SE of estimates

Gradient ∇f → n×1 vector → 1st order, scalar function
Jacobian J → m×n matrix → 1st order, vector function
Hessian H → n×n matrix → 2nd order, curvature

H positive definite → local minimum
H negative definite → local maximum
H mixed signs → saddle point

Core Idea

ELI5: You have a belief about something. You see new evidence. You update your belief. That's it.

Posterior ∝ Likelihood × Prior

P(hypothesis | data) ∝ P(data | hypothesis) × P(hypothesis)

- **Prior** — what you believed *before* seeing data
- **Likelihood** — how probable the data is given a hypothesis
- **Posterior** — your updated belief *after* seeing data

Simple Example

You test positive for a rare disease (1% of population). Test is 90% accurate.

Prior: P(sick) = 0.01
Likelihood: P(positive | sick) = 0.90
P(positive | healthy) = 0.10

Posterior: P(sick | positive) = (0.90 × 0.01) / P(positive)
≈ 8.3%
Despite a positive test, you're likely still healthy — because the disease is rare. This is the base rate trap, and Bayes handles it naturally.

Frequentist vs Bayesian

	Frequentist	Bayesian
Parameters	Fixed, unknown constants	Random variables with distributions
Output	Point estimate + p-value	Full posterior distribution
Uncertainty	Confidence intervals	Credible intervals
Prior knowledge	Ignored	Explicitly incorporated

Key Models to Know
Naive Bayes — classification using Bayes' theorem, assumes feature independence. Fast, works well on text.
Bayesian Linear Regression — instead of one weight vector, you get a *distribution* over weights. Naturally quantifies uncertainty.
Gaussian Processes — non-parametric Bayesian model. Gives a distribution over functions, not just predictions. Used in hyperparameter tuning (Bayesian optimization).
Probabilistic Graphical Models (PGMs) — encode conditional dependencies between variables (Bayesian Networks, HMMs).

Interview Phrases
"Bayesian inference lets us quantify uncertainty in our estimates, not just give point predictions. This matters in production when you need to know not just what the model predicts, but how confident it is."
"The prior is a regularizer — a strong prior shrinks parameters toward a belief, just like L2 regularization shrinks weights toward zero."

Cheat Sheet

Term	Meaning
Prior P(θ)	Belief before data
Likelihood P(data|θ)	How well θ explains the data
Posterior P(θ|data)	Updated belief after data

MAP estimate	Mode of the posterior (like regularized MLE)
MLE	Maximum likelihood — no prior, just fit the data
Credible interval	95% of posterior mass — the Bayesian CI

Layers in a Neural Network

1. Fully Connected (Dense)

Every neuron connects to every neuron in the next layer.

output = activation(W·x + b)

W = weight matrix b = bias x = input vector

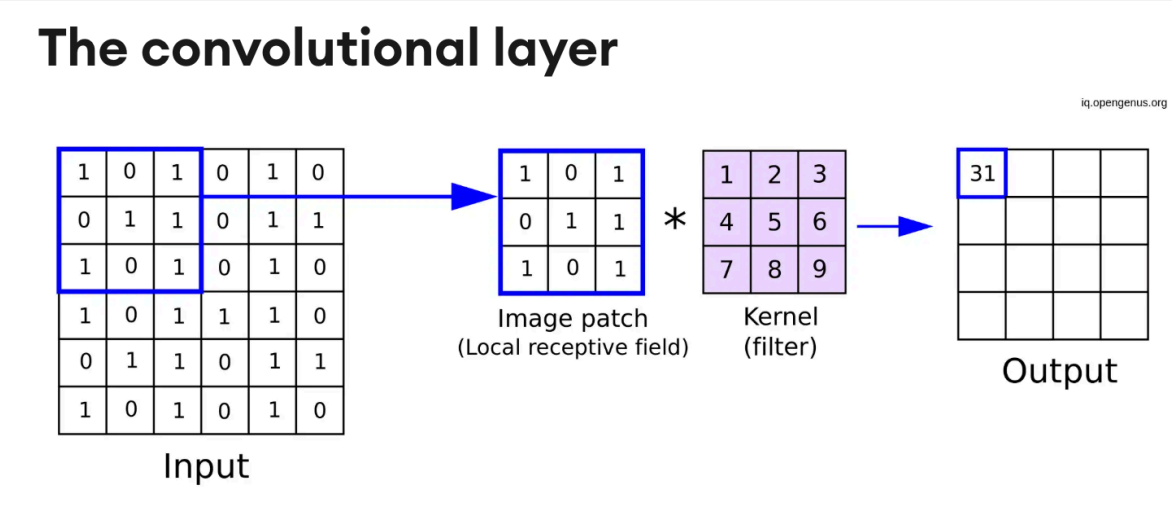
Use: classification heads, final layers, tabular data. **Problem:** doesn't scale to images — too many parameters.

2. Convolutional (CNN)

A small filter (kernel) slides over the input, detecting local patterns. Same filter reused everywhere — that's *weight sharing*.

Filter 3×3 slides over image → detects edges, textures, shapes
Early layers → edges
Middle layers → shapes
Deep layers → faces, objects

Use: images, time series, any data with local structure. **Key params:** kernel size, stride, padding, number of filters.



https://www.superannotate.com/blog/guide-to-convolutional-neural-networks

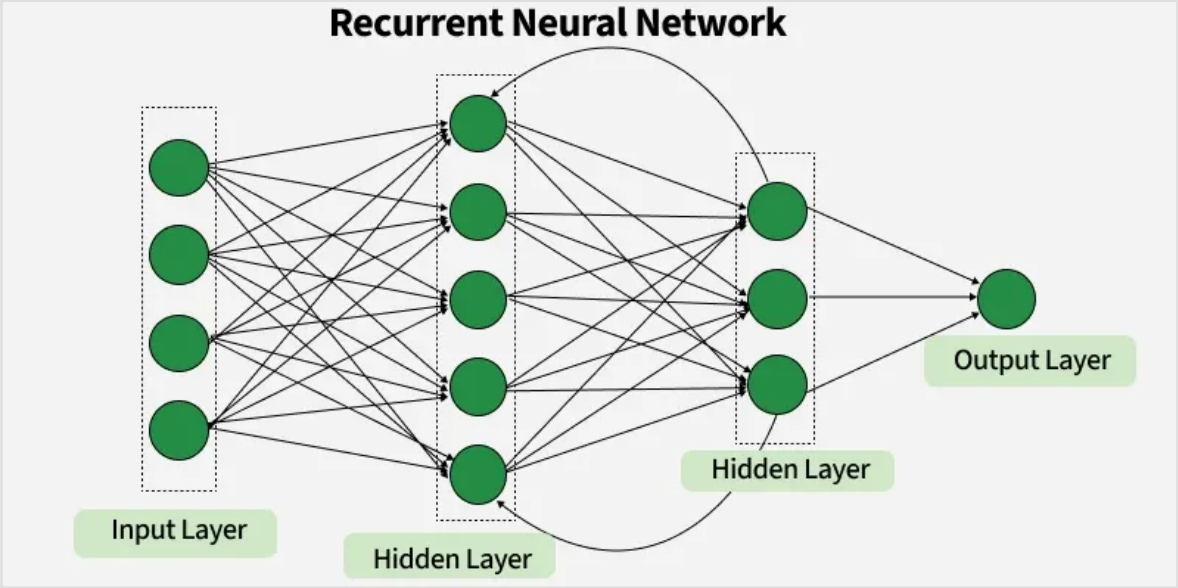
3. Recurrent (RNN / LSTM / GRU)

Has a hidden state — a memory that carries information across time steps.

$h_t = f(W_x \cdot x_t + W_h \cdot h_{t-1} + b)$

↑ current input ↑ previous memory

Vanilla RNN — suffers from vanishing gradients (forgets early steps). **LSTM** — adds gates (forget, input, output) to control memory. Solves vanishing gradient.
GRU — simpler than LSTM, fewer parameters, similar performance.
Use: text, speech, time series sequences.



Intuition

A recurrent network processes sequences step by step.
At each step it keeps a summary of what it has seen so far:

```
previous hidden state + current input → new hidden state
```

That hidden state is the network's memory.

Why Is It Called "Hidden"?

Because it is:

- **internal to the model**
- not directly part of the input/output
- a latent representation learned during training

So "hidden" means:
Internal representation not directly observed

Model	Purpose	Advantages	Disadvantages	Better / Modern Alternatives
Vanilla RNN	Basic sequential modeling	Very simple, lightweight, easy to learn	Severe vanishing/ exploding gradients, poor long-term memory	GRU, LSTM
LSTM	Capture long-term dependencies in sequences	Strong memory mechanism, robust, proven in many domains	More parameters, slower training/ inference, sequential (not parallelizable)	GRU (lighter), Transformer, TCN
GRU	Efficient long/ medium-range sequence modeling	Similar performance to LSTM with fewer parameters, faster	Sometimes slightly less expressive than LSTM	Transformer, TCN
Bidirectional LSTM/GRU	Use past + future context for each token/ time step	Better context understanding, improved accuracy in offline tasks	Cannot be used causally/ online, doubles compute	Transformer encoders
Seq2Seq RNN/LSTM	Map input sequences to output sequences	Flexible encoder-decoder structure	Bottlenecked by fixed hidden state, weaker than attention methods	Transformer Seq2Seq

Temporal Convolutional Network (TCN)	Sequence modeling with dilated convolutions	Parallelizable, stable gradients, handles long receptive fields well	Less intuitive memory mechanism, architecture tuning can be tricky	Transformer, SSMs (depending on task)
Transformer	Attention-based global sequence modeling	Excellent long-range modeling, parallel training, SOTA in many tasks	Computationally expensive, memory-heavy, data-hungry	Long-context transformer variants, SSMs
State Space Models (e.g. Mamba/S4)	Efficient very-long-sequence modeling	Scales better on long contexts, efficient inference	Newer/less mature ecosystem, more specialized	Transformer (if compute not an issue)

4. Dropout

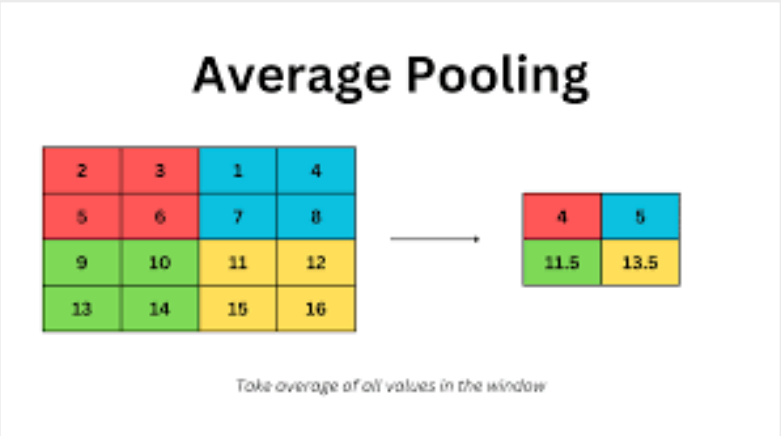
During training, randomly sets a fraction of neurons to zero. Forces the network to not rely on any single neuron.
he primary purpose of dropout in neural networks is to **prevent overfitting** by acting as a regularization technique, forcing the network to learn more robust and generalizable features

Training: each neuron active with probability p (e.g. 0.8)
Inference: all neurons active, weights scaled by p
It's a regularizer — reduces overfitting. Equivalent to training an ensemble of subnetworks.
Typical values: 0.2–0.5. Don't use on small datasets or final layers.

5. Pooling

Downsamples the spatial dimensions. Reduces computation and adds position invariance.

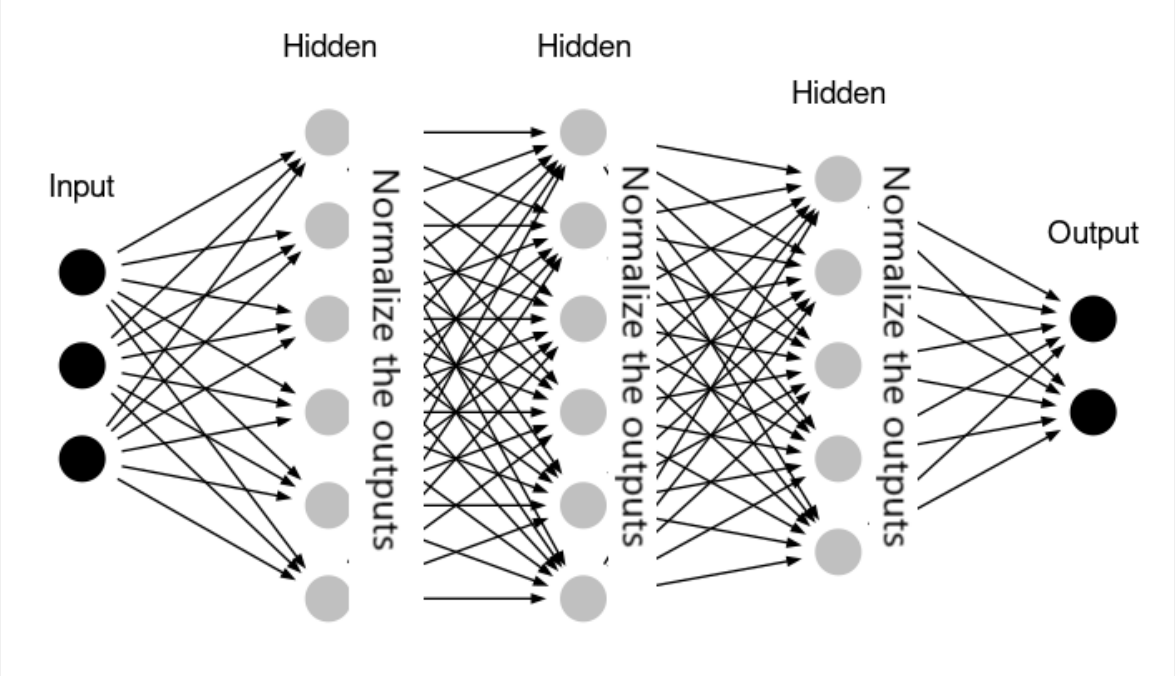
Max Pooling 2×2: takes the MAX value in each 2×2 region
Avg Pooling 2×2: takes the AVERAGE value in each 2×2 region
Max pooling — keeps the strongest signal (most common). **Global average pooling** — collapses entire feature map to one number per channel. Used before final dense layer.



6. Batch Normalization

Normalizes the activations of each layer across the mini-batch. Applied *before or after* the activation function.

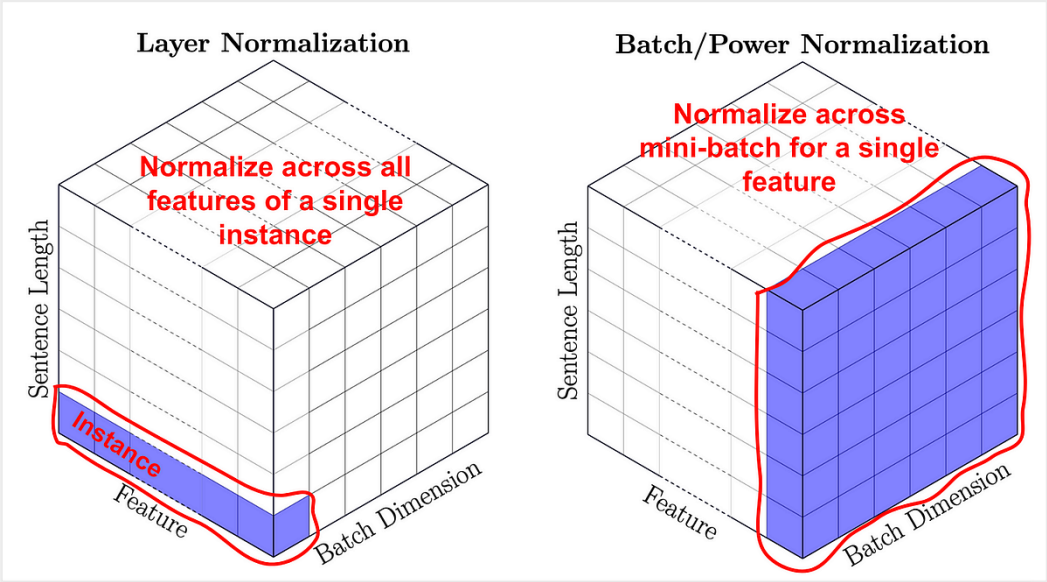
$\hat{x} = (x - \mu_{batch}) / \sigma_{batch}$ ← normalize
$y = \gamma \cdot \hat{x} + \beta$ ← scale and shift (learned)



Why it helps:

- Reduces internal covariate shift
- Allows higher learning rates
- Acts as mild regularizer
- Makes training much more stable

Use: almost always after Conv or Dense layers in deep networks.



7. Layer Normalization

Like Batch Norm but normalizes *across features* for a single sample, not across the batch.

Batch Norm: normalize across the batch dimension
Layer Norm: normalize across the feature dimension
Use: Transformers and RNNs — where batch size may be 1 or sequences vary in length. Batch Norm breaks in those settings; Layer Norm doesn't.

8. Attention

Lets the model focus on the most relevant parts of the input dynamically, instead of treating all positions equally.

Look at all parts of the sequence and weigh which ones matter most right now.

In One Sentence
Attention = dynamic focus over relevant information

$$\text{Attention}(Q, K, V) = \text{softmax}(Q \cdot K^T / \sqrt{d_k}) \cdot V$$

Q = Query "what am I looking for?"
K = Key "what do I have?"
V = Value "what do I return?"
Self-attention — each token attends to every other token in the sequence. Foundation of Transformers. **Multi-head attention** — run attention h times in parallel, each learning different relationships.
ELI5: Translating "bank" — attention looks at surrounding words to decide if it

means river bank or financial bank.

Self-attention allows the model to consider all positions in the input sequence when producing the output for a specific position. The most widely known example of this is the Transformer model, which uses self-attention to process sequences in parallel, unlike traditional RNNs or LSTMs.

Quick Cheat Sheet

Layer	Purpose	Typical Use
Fully Connected	Global transformation	Tabular, final layers
Convolutional	Local pattern detection	Images, signals
Recurrent (LSTM/ GRU)	Sequential memory	Text, time series
Dropout	Regularization	Prevent overfitting
Pooling	Downsample + invariance	After conv layers
Batch Norm	Stabilize activations across batch	Deep CNNs
Layer Norm	Stabilize activations across features	Transformers, RNNs
Attention	Dynamic relevance weighting	NLP, Transformers

Learning Schemas / Optimization

Core Idea

The optimizer decides **how** to update weights after computing the gradient. The gradient tells you the direction — the optimizer decides the step size and strategy.

Basic update: $w \leftarrow w - \alpha \cdot \nabla L$

α = learning rate ∇L = gradient of loss

1. SGD — Stochastic Gradient Descent

Compute gradient on a **random mini-batch**, update weights.

$w \leftarrow w - \alpha \cdot \nabla L(\text{mini-batch})$

Vanilla SGD problems:

- Same learning rate for all parameters

- Slow in flat regions, oscillates in narrow valleys
- Sensitive to learning rate choice

SGD + Momentum — adds a velocity term, accumulates past gradients. Smooths oscillations, speeds up flat regions.

$v \leftarrow \beta \cdot v + \nabla L \qquad (\beta \approx 0.9)$
$w \leftarrow w - \alpha \cdot v$

ELI5: Like a ball rolling downhill — momentum carries it through flat spots and small bumps.

2. AdaGrad

Adapts learning rate **per parameter** — parameters updated frequently get smaller steps, rare ones get bigger steps.

$G \leftarrow G + (\nabla L)^2 \qquad$ (accumulated squared gradients)
$w \leftarrow w - \alpha / \sqrt{G} \cdot \nabla L$

Problem: G keeps growing → learning rate shrinks to near zero and training stops. Bad for deep networks.
Good for: sparse data, NLP with rare words.

3. RMSProp

Fixes AdaGrad by using an **exponential moving average** of squared gradients instead of accumulating forever.

$G \leftarrow \beta \cdot G + (1-\beta) \cdot (\nabla L)^2 \qquad (\beta \approx 0.9$, decaying average)
$w \leftarrow w - \alpha / \sqrt{G} \cdot \nabla L$

Learning rate stays alive throughout training. Good for RNNs.

4. Adam — Adaptive Moment Estimation

Combines **momentum** (1st moment) + **RMSProp** (2nd moment). Most popular optimizer in practice.

$m \leftarrow \beta_1 \cdot m + (1-\beta_1) \cdot \nabla L \qquad$ (1st moment — mean of gradients)
$v \leftarrow \beta_2 \cdot v + (1-\beta_2) \cdot (\nabla L)^2 \qquad$ (2nd moment — variance of gradients)

$\hat{m} = m / (1 - \beta_1^t) \qquad$ (bias correction)
$\hat{v} = v / (1 - \beta_2^t)$

$w \leftarrow w - \alpha \cdot \hat{m} / (\sqrt{\hat{v}} + \varepsilon)$

Defaults: β_1=0.9, β_2=0.999, ε=1e-8, α=1e-3
ELI5: Adam remembers which direction it's been going (momentum) AND how bumpy each parameter's landscape is (variance). It takes big steps on smooth parameters, small steps on noisy ones.

Adam is essentially RMSprop with added momentum.

5. AdamW

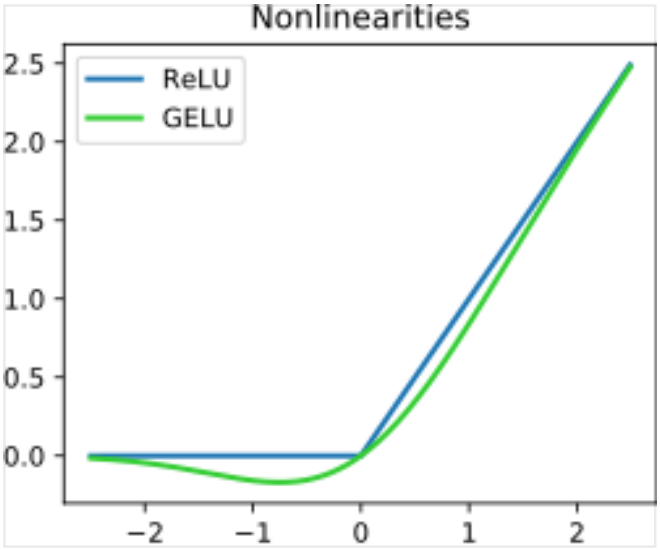
Adam + proper **weight decay** (L2 regularization decoupled from the gradient update). Fixes a subtle bug in Adam where L2 reg doesn't behave correctly.

$w \leftarrow w - \alpha \cdot (\hat{m} / (\sqrt{\hat{v}} + \varepsilon) + \lambda \cdot w)$

Default choice for Transformers and LLMs. Almost always better than vanilla Adam.

Optimizer Comparison

Optimizer	Adaptive LR	Momentum	Best For
SGD	No	Optional	CV models, fine-tuned control
SGD + Momentum	No	Yes	ImageNet-scale CNNs
AdaGrad	Yes	No	Sparse features, NLP
RMSProp	Yes	No	RNNs
Adam	Yes	Yes	General purpose, fast prototyping
AdamW	Yes	Yes	Transformers, LLMs



$$\mathrm{ReLU}(x) = x^+ = \max(0, x) = \frac{x + |x|}{2} = \begin{cases} x & \text{if } x > 0, \\ 0 & x \leq 0 \end{cases}$$

6. Batch Size vs Learning Rate Tradeoff

Small batch (8–64):

- Noisy gradients → acts as regularizer
- Escapes local minima more easily
- Slower per epoch, but often better generalization

Large batch (512–4096):

- Stable, accurate gradients
- Faster training (parallelism)
- Tends to converge to sharp minima → worse generalization
- Needs a **larger learning rate** to compensate

Linear scaling rule:
If you multiply batch size by k → multiply learning rate by k

Batch 256, lr=0.1 → Batch 1024, lr=0.4

7. Learning Rate — The Most Important Hyperparameter

Too high → loss explodes, overshoots minimum **Too low** → training is slow, gets stuck

Learning Rate Schedules

Step decay — reduce LR by factor every N epochs

α ← α × 0.1 every 30 epochs

Cosine annealing — smooth decay following a cosine curve. Very common, works well.

$\alpha_t = \alpha_{min} + ½(\alpha_{max} - \alpha_{min})(1 + \cos(\pi t/T))$
Warmup — start with very small LR, ramp up, then decay. Standard for Transformers.

Epochs 0→5: LR ramps up (warmup)
Epochs 5→N: LR decays (cosine or linear)
Cyclical LR — oscillates between min and max. Can escape local minima.

8. Gradient Problems & Fixes

Problem	Symptom	Fix
Vanishing gradient	Early layers don't learn	ReLU, residual connections, LSTM gates
Exploding gradient	Loss goes to NaN	Gradient clipping (clip_grad_norm)
Dying ReLU	Neurons output 0 forever	Leaky ReLU, ELU, proper init
Saddle points	Training stalls	Momentum, Adam

Quick Cheat Sheet

Concept	One-liner
SGD	Simple, needs tuning, great final performance
Momentum	Smooths gradient direction over time
Adam	Adaptive LR per param + momentum, go-to default
AdamW	Adam + correct weight decay, use for Transformers
Large batch	Stable but sharp minima, scale LR proportionally
LR warmup	Prevent early instability in Transformers
Gradient clipping	Cap gradient norm to prevent explosion

Interview tip: *"In practice: start with Adam or AdamW, learning rate 1e-3, add warmup if using a Transformer. If you need the best possible generalization and have time to tune, SGD + momentum + cosine decay often beats Adam on vision tasks — that's why ResNet papers use it."*

PART 1 — Loss Functions

The Job of a Loss Function

Measures how wrong the model is. The optimizer minimizes it. Your choice of loss **encodes your assumptions** about the problem.

1. L-Based Distance Losses (Regression)

MSE — Mean Squared Error (L2 Loss)

$$L = (1/n) \Sigma (y_i - \hat{y}_i)^2$$

- Penalizes large errors heavily (squaring amplifies outliers)
- Smooth gradient → easy to optimize
- **Use when:** outliers are rare and should be penalized hard

MAE — Mean Absolute Error (L1 Loss)

$$L = (1/n) \Sigma |y_i - \hat{y}_i|$$

- Treats all errors linearly, robust to outliers
- Gradient is constant → can oscillate near minimum
- **Use when:** outliers are common (house prices, demand forecasting)

Huber Loss (smooth L1)

$$L = (1/2)(y - \hat{y})^2 \quad \text{if } |y - \hat{y}| \le \delta$$
$$L = \delta \cdot |y - \hat{y}| - (1/2)\delta^2 \quad \text{otherwise}$$

- L2 for small errors, L1 for large errors — best of both
- δ is a tunable threshold
- **Use when:** you want robustness but smooth gradients

RMSE — just $\sqrt{}$MSE. Same behavior as MSE, but interpretable in original units.

Loss	Outlier Sensitivity	Gradient	Use Case
MSE	High	Smooth	Clean data, Gaussian noise
MAE	Low	Constant	Noisy data, robust regression
Huber	Medium	Smooth	General purpose regression

Credible intervals (Bayesian) provide a direct probability range for a parameter based on data and prior beliefs (e.g., "there is a 95% chance the true value is

here"). Confidence intervals (Frequentist) represent the frequency with which an interval calculated from repeated, random sampling contains the fixed true parameter

Loss Functions & Evaluation / Bias-Variance Tradeoff

PART 1 — Loss Functions

The Job of a Loss Function

Measures how wrong the model is. The optimizer minimizes it. Your choice of loss **encodes your assumptions** about the problem.

1. L-Based Distance Losses (Regression)

MSE — Mean Squared Error (L2 Loss)

$$L = (1/n) \Sigma (y_i - \hat{y}_i)^2$$

- Penalizes large errors heavily (squaring amplifies outliers)
- Smooth gradient → easy to optimize
- **Use when:** outliers are rare and should be penalized hard

MAE — Mean Absolute Error (L1 Loss)

$$L = (1/n) \Sigma |y_i - \hat{y}_i|$$

- Treats all errors linearly, robust to outliers
- Gradient is constant → can oscillate near minimum
- **Use when:** outliers are common (house prices, demand forecasting)

Huber Loss (smooth L1)

$$L = (1/2)(y - \hat{y})^2 \quad \text{if } |y - \hat{y}| \le \delta$$
$$L = \delta \cdot |y - \hat{y}| - (1/2)\delta^2 \quad \text{otherwise}$$

- L2 for small errors, L1 for large errors — best of both
- δ is a tunable threshold
- **Use when:** you want robustness but smooth gradients

RMSE — just √MSE. Same behavior as MSE, but interpretable in original units.

Loss	Outlier Sensitivity	Gradient	Use Case
MSE	High	Smooth	Clean data, Gaussian noise
MAE	Low	Constant	Noisy data, robust regression
Huber	Medium	Smooth	General purpose regression

Entropu

$$H = -\sum p(x)\log p(x)$$

Cross entropy ,:

2. Cross-Entropy (Classification)

Cross-entropy loss is the standard loss function for classification tasks in machine learning, used to measure the difference between predicted probability distributions and true labels.

Binary Cross-Entropy

L = -[y·log(ŷ) + (1-y)·log(1-ŷ)]

y = true label (0 or 1) ŷ = predicted probability

- Heavily penalizes confident wrong predictions (log blows up near 0)
- **Use:** logistic regression, binary classifiers, last layer sigmoid

Categorical Cross-Entropy

L = -Σ y_i · log($\hat{y}_i$) (sum over classes)

- y_i is one-hot encoded, so only the true class term survives
- **Use:** multi-class classification, last layer softmax

ELI5: If you're 99% sure it's a cat and it's actually a dog, cross-entropy punishes you enormously. If you said 60% cat, punishment is moderate. It rewards calibrated confidence.

KL Divergence — related to cross-entropy, measures how much distribution P differs from Q.

KL(P‖Q) = Σ P(x) · log(P(x)/Q(x))

Cross-entropy H(P,Q) = H(P) + KL(P‖Q)

Use: VAEs, knowledge distillation, distribution matching.

3. Other Important Losses

Hinge Loss (SVM)

$L = \max(0, 1 - y \cdot \hat{y}) \qquad y \in \{-1, +1\}$

- Zero loss if prediction is correct AND confident (margin > 1)
- **Use:** SVMs, max-margin classifiers

Focal Loss

$L = -\alpha(1 - \hat{y})^{\gamma} \cdot \log(\hat{y})$

- Down-weights easy examples, focuses on hard ones
- **Use:** class imbalance problems (object detection, fraud detection)

Contrastive / Triplet Loss

$L = \max(0, d(\text{anchor}, \text{positive}) - d(\text{anchor}, \text{negative}) + \text{margin})$

- Pulls similar samples together, pushes different ones apart
- **Use:** embeddings, face recognition, recommendation systems

ELBO (Evidence Lower Bound)

$L = E[\log P(x|z)] - KL(Q(z|x) \,\|\, P(z))$
$= \text{reconstruction loss} + \text{regularization}$

- **Use:** Variational Autoencoders (VAEs)

PART 2 — Evaluation Metrics

Regression Metrics

MAE = mean absolute error → interpretable, robust
RMSE = root mean squared error → penalizes large errors
R^2 = 1 - SS_res/SS_tot → % variance explained (1 = perfect)
MAPE = mean(|y - ŷ| / |y|) × 100 → % error, breaks if y ≈ 0

Classification Metrics

Accuracy = correct / total → misleading on imbalanced data

Precision = TP / (TP + FP) → "of predicted positives, how many real?"

Recall = TP / (TP + FN) → "of real positives, how many caught?"
F1 = 2 · P·R / (P + R) → harmonic mean, balances both

AUC-ROC = area under TPR vs FPR curve
0.5 = random, 1.0 = perfect
Good for imbalanced classes, threshold-independent

AUC-PR = area under Precision-Recall curve
Better than ROC when positives are very rare

When to use what:

- Fraud detection → Recall (missing fraud is costly)
- Spam filter → Precision (false positives annoy users)
- Medical diagnosis → F1 or AUC-PR
- Imbalanced classes → AUC-ROC or AUC-PR, never raw accuracy

PART 3 — Bias / Variance Tradeoff

Core Idea

Total prediction error has three components:

Error = Bias² + Variance + Irreducible Noise

Bias → error from wrong assumptions (underfitting)
Variance → error from sensitivity to training data (overfitting)
Noise → inherent randomness, can't be reduced

ELI5: Bias is a consistently bad aim — your arrows always land left of target. Variance is inconsistent aim — your arrows scatter everywhere. You want both low. The tradeoff is: reducing one often increases the other.

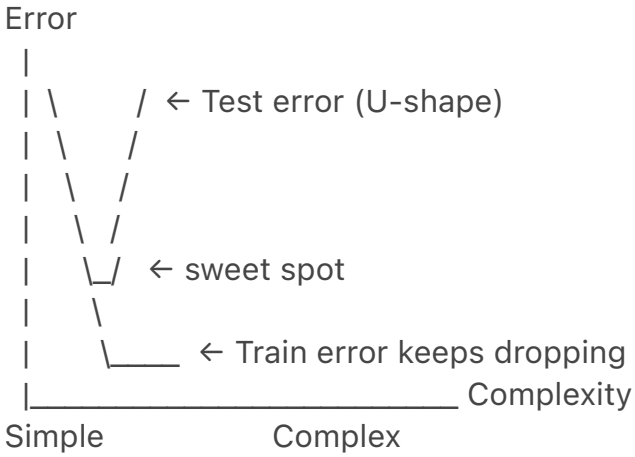
Intuition

High Bias, Low Variance → underfitting
Model too simple, misses real patterns
Training error HIGH, Test error HIGH
Fix: more complexity, more features, less regularization

Low Bias, High Variance → overfitting
Model memorizes training data, fails on new data
Training error LOW, Test error HIGH
Fix: more data, regularization, dropout, simpler model

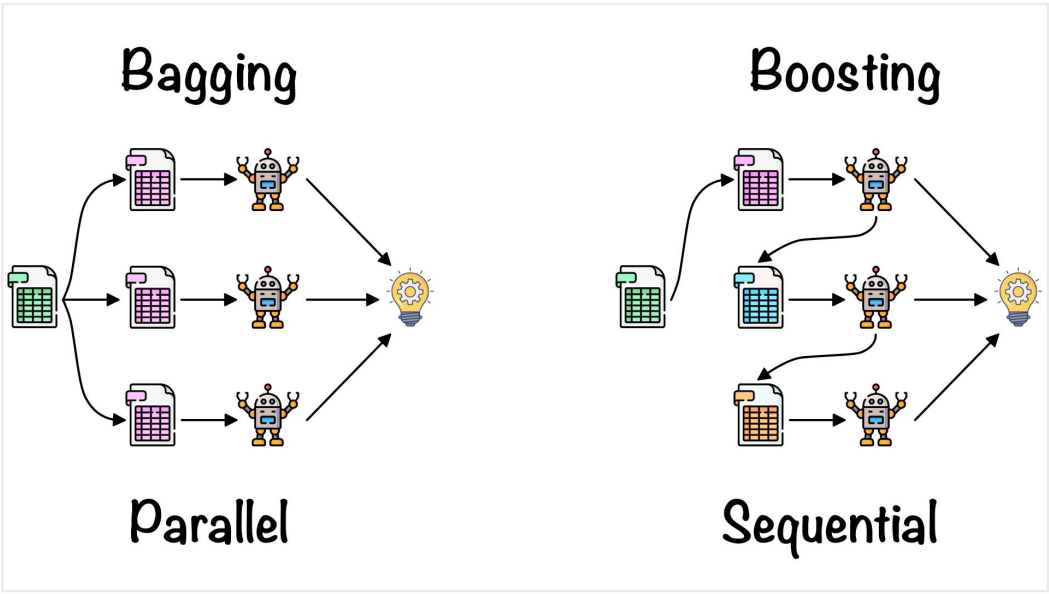
Sweet spot → generalizes well
Training error LOW, Test error LOW (and close together)

Model Complexity vs Error



Bias-Variance in Common Models

Model	Bias	Variance	Notes
Linear Regression	High	Low	Underfits complex data
Deep Neural Net	Low	High	Needs regularization
Decision Tree (full)	Low	High	Overfits easily
Random Forest	Low	Medium	Averaging reduces variance
Bagging	Same	Lower	Reduces variance by averaging
Boosting	Lower	Same	Reduces bias by correcting errors



How to Diagnose

High bias signals:
- Training error is high
- Train error ≈ test error (both bad)
- Learning curves plateau at high error

High variance signals:
- Training error is low
- Large gap between train and test error
- Performance varies a lot across folds

Fixes Summary

Problem	Fix
High bias	More features, deeper model, less regularization
High variance	More data, dropout, regularization (L1/L2), simpler model
Both	Better architecture, cross-validation, feature engineering

Quick Cheat Sheet

Concept	One-liner
MSE	Punishes big errors hard, sensitive to outliers
MAE	Robust, treats all errors equally

Cross-entropy	Punishes confident wrong predictions exponentially
Focal loss	Cross-entropy that focuses on hard examples
Bias	Systematic error — model too simple
Variance	Random error — model too sensitive to training data
Overfitting	Low train error, high test error → high variance
Underfitting	High train error, high test error → high bias

Interview tip: *"Bias and variance are complementary. Regularization increases bias but decreases variance — that's the explicit tradeoff. Ensemble methods like Random Forest attack variance directly by averaging, while boosting attacks bias by iteratively correcting mistakes."*

Summary of Differences

Feature	Epoch Graph	Dataset Size Graph
X-axis	Number of epochs (passes through the data)	Size of the unique training dataset
Purpose	To evaluate training progress, diagnose overfitting/ underfitting, and tune the number of epochs	To evaluate data efficiency, identify performance saturation, and estimate data needs
Metric Plotted	Loss/Accuracy (training and validation)	Accuracy/Performance (typically on a test set)
Resulting Insight	When to stop training (number of epochs)	If more unique data is needed to improve the model

Dataset Size Graph (Performance vs. Dataset Size)

- **Purpose:** The primary use of this graph (often called a **learning curve based on data size**) is to **evaluate the efficiency of the model architecture relative to the amount of training data** and determine how much data is necessary to reach optimal performance.
- **X-axis:** The size (or number of unique data points) of the training

dataset used. This measures unique data points seen once, regardless of how many epochs are run.
- **Y-axis:** A performance metric, such as accuracy or error rate.
- **Analysis:**
  - Performance typically increases as the dataset size grows and eventually saturates.
  - If the performance saturates, adding more data is unlikely to significantly improve the model further, suggesting the model has learned the general distribution of the data well.
  - This graph helps compare the *data efficiency* of different algorithms by showing how much data each requires to reach a certain performance level

Linear Separability & Logistic Regression

Short Answer
Linear separability is **not a requirement** — it is **not even desirable** to assume it. But it does affect behavior in important ways.

What Logistic Regression Actually Does
Logistic regression models the **probability** of a class, not a hard boundary. It finds the linear decision boundary that best separates classes *in probability space*:

P(y=1 | x) = σ(w·x + b) = 1 / (1 + e^(-w·x - b))
It doesn't need perfect separation — it needs a **linear relationship between features and log-odds**:

log(P(y=1) / P(y=0)) = w·x + b ← this is what must be linear

What Happens When Data IS Linearly Separable
This is actually a **problem**, not a benefit:

If classes are perfectly separable →
  weights w grow toward infinity
  sigmoid pushes probabilities to 0 and 1
  model never converges
  coefficients are undefined (MLE doesn't exist)
The optimizer keeps increasing weights forever trying to make the boundary sharper. You must use **regularization (L2)** to constrain this — which is why sklearn's LogisticRegression has C (inverse regularization) on by default.

What Happens When Data is NOT Linearly Separable

This is the **normal, expected case**. Logistic regression handles it fine:

- It finds the best linear boundary it can
- Outputs calibrated probabilities, not just hard labels
- Overlapping classes → probabilities stay away from 0/1, which is honest and useful

When Logistic Regression Struggles

Not because of separability, but because the **true decision boundary is non-linear**. Fix with:

- Feature engineering (x^2, $x \cdot y$, $\log(x)$)
- Kernelized logistic regression
- Switch to a non-linear model (tree, NN)

Interview phrasing: *"Linear separability is neither required nor ideal for logistic regression. Perfect separability actually causes the MLE to fail — weights diverge. The real assumption is that the log-odds are linear in the features, which is a much weaker condition."*

SVMs — Support Vector Machines

Core Idea

Find the hyperplane that separates classes with the **maximum margin** — the widest possible gap between the two classes. Only the points closest to the boundary matter — these are the **support vectors**.

Maximize: $\text{margin} = 2 / \|w\|$
Subject to: $y_i(w \cdot x_i + b) \geq 1$ for all i

ELI5: Draw a line between two groups. Now push that line as far as possible from both groups until it's perfectly centered in the gap. The margin is that gap width.

Hard vs Soft Margin

Hard margin — requires perfect separation. Fails if any overlap exists.
Soft margin — allows some misclassifications, controlled by parameter C:

C large → narrow margin, fewer errors, more overfitting
C small → wide margin, more errors allowed, more regularization

The Kernel Trick

SVMs can find **non-linear boundaries** without explicitly computing new features — by mapping data to higher dimensions implicitly via a kernel

function:

Linear kernel: K(x,z) = x·z
Polynomial kernel: K(x,z) = (x·z + c)^d
RBF / Gaussian: K(x,z) = exp(-γ‖x-z‖²) ← most common
ELI5: Data not separable in 2D? Project it into 100D where it is — but do the math without actually going there. That's the kernel trick.

Pros

- Works well in **high-dimensional spaces** (text, genomics)
- Effective when **features > samples**
- The kernel trick gives non-linear power with convex optimization (no local minima)
- Only support vectors matter → **memory efficient** for inference
- Strong theoretical guarantees (margin maximization, VC theory)

Cons

- **Doesn't scale** — training is O(n²) to O(n³), painful beyond ~100k samples
- **No probability output** natively — needs Platt scaling as a post-hoc fix, which is slow
- Kernel and C choice requires careful tuning
- Hard to interpret — especially with RBF kernel
- **Doesn't handle noise well** with hard margin
- Multi-class requires workarounds (one-vs-one or one-vs-rest)

SVM vs Logistic Regression

	SVM	Logistic Regression
Objective	Maximize margin	Maximize likelihood
Output	Decision boundary	Calibrated probabilities
Outlier sensitivity	Low (only SVs matter)	High (all points contribute)
Non-linearity	Kernel trick	Feature engineering
Scalability	Poor (large n)	Good
Interpretability	Low (RBF)	High

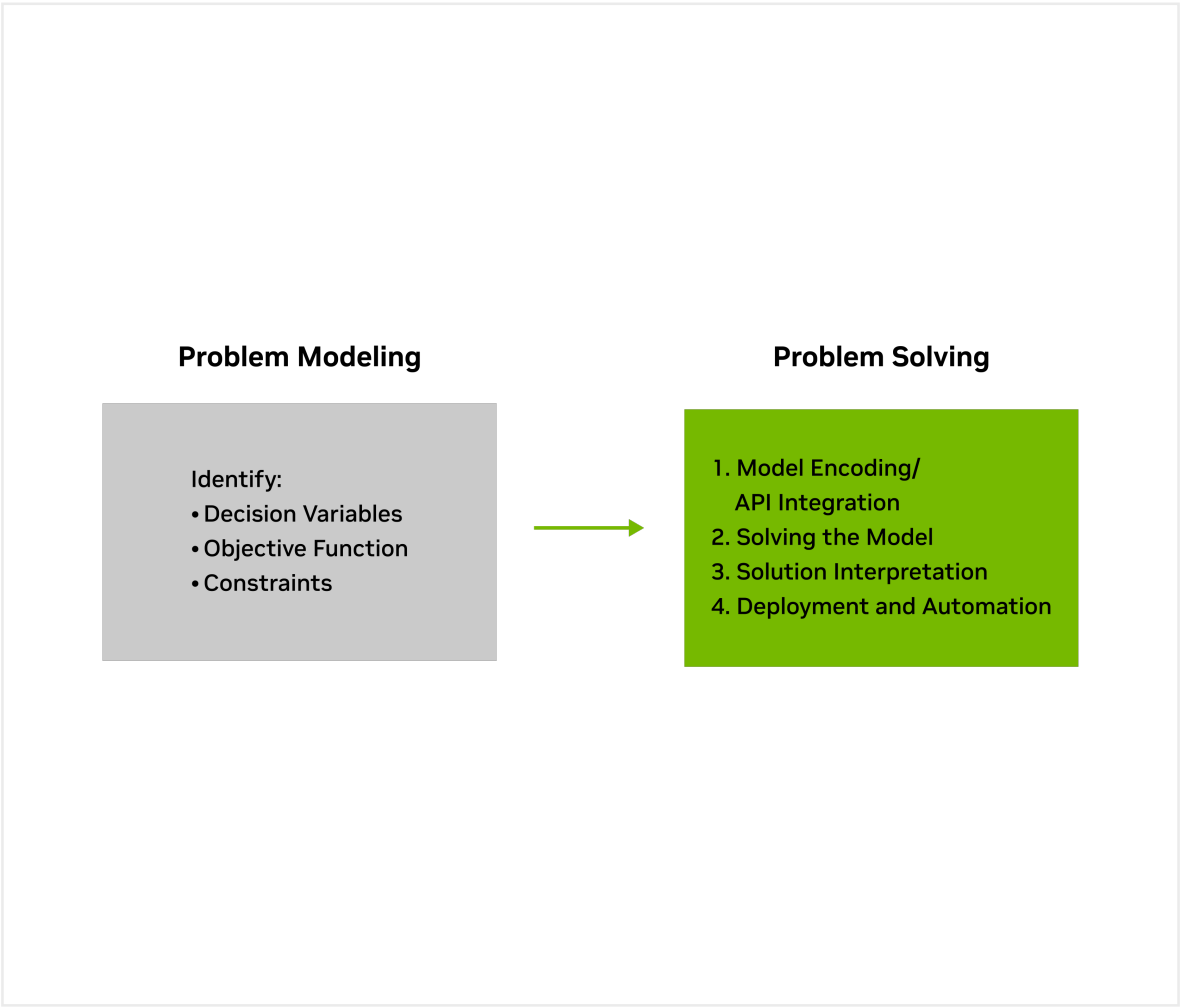
When to Actually Use SVMs Today

SVMs are largely replaced by gradient boosting (XGBoost) and neural nets for most tasks. Still relevant for:

- Small datasets with high-dimensional features (text classification, bioinformatics)
- When you need a convex optimization guarantee
- When labeled data is scarce

Interview tip: *"SVMs and logistic regression both find linear boundaries, but with different objectives. SVMs maximize the geometric margin — making them less sensitive to individual points. The kernel trick is SVMs' biggest strength, but their $O(n^3)$ training complexity is their biggest weakness at scale."*

Feature	Pros (Advantages)	Cons (Disadvantages)
Data Dimensionality	Highly effective in **high-dimensional spaces**.	Performance can degrade if the number of features is much greater than the number of samples.
Data Size	Works well with **small to medium-sized** datasets.	Not suitable for very **large datasets** due to high training time and computational cost.
Complexity	Uses the **kernel trick** to handle complex, non-linear relationships.	Choosing the **right kernel** and parameters (like and) can be difficult and time-consuming.



A **heuristic**[1] or **heuristic technique** (*problem solving*, *mental shortcut*, *rule of thumb*)[2][3][4][5] is any approach to problem solving that employs a pragmatic method that is not fully optimized, perfected, or rationalized, but is nevertheless "good enough" as an approximation or attribute substitution.[6][7]

Branch-and-bound (**BB**, **B&B**, or **BnB**) is a method for solving optimization problems by breaking them down into smaller subproblems and using a bounding function to eliminate subproblems that cannot contain the optimal solution.

In mathematical optimization, the **cutting-plane method** is any of a variety of optimization methods that iteratively refine a feasible set or objective function by means of linear inequalities, termed *cuts*. Such procedures are commonly used to find integer solutions to mixed integer linear programming (MILP) problems, as well as to solve general, not necessarily differentiable convex optimization problems.

Optimization problems [edit]

Main article: Optimization problem

Optimization problems can be divided into two categories, depending on whether the variables are continuous or discrete:

- An optimization problem with discrete variables is known as a *discrete optimization*, in which an object such as an integer, permutation or graph must be found from a countable set.
- A problem with continuous variables is known as a *continuous optimization*, in which optimal arguments from a continuous set must be found. They can include constrained problems and multimodal problems.

An optimization problem can be represented in the following way:

Given: a function $f : A \to \mathbb{R}$ from some set A to the real numbers
Sought: an element $\mathbf{x}_0 \in A$ such that $f(\mathbf{x}_0) \leq f(\mathbf{x})$ for all $\mathbf{x} \in A$ ("minimization") or such that $f(\mathbf{x}_0) \geq f(\mathbf{x})$ for all $\mathbf{x} \in A$ ("maximization").

En los problemas de optimización, el método de los **multiplicadores de Lagrange**, llamados así en honor a Joseph Louis Lagrange, es un procedimiento para encontrar los máximos y mínimos relativos (o locales) de funciones de múltiples variables sujetas a restricciones.[1]

Los multiplicadores de Lagrange son un **método de optimización para encontrar máximos y mínimos locales de funciones de varias variables sujetas a restricciones de igualdad**. La técnica introduce nuevas variables (λ, los multiplicadores) para reducir el problema restringido a un sistema de ecuaciones sin restricciones, donde el gradiente de la función objetivo es proporcional al gradiente de la restricción ($\nabla f = \lambda \nabla g$). Wikipedia +3

Conceptos Clave

- **Objetivo:** Maximizar o minimizar una función objetivo $f(x, y, ...)$ sujeta a una restricción $g(x, y, ...) = c$.
- **El Lagrangiano:** Se define una nueva función, la función de Lagrange, definida como $\mathcal{L}(x, y, \lambda) = f(x, y) - \lambda(g(x, y) - c)$.
- **Sistema de Ecuaciones:** Se buscan los puntos críticos calculando las derivadas parciales de la función de Lagrange con respecto a todas las variables (x, y, λ) e igualándolas a cero:
  - $\nabla f = \lambda \nabla g$
  - $g(x, y) = c$
- **Significado de λ:** El multiplicador de Lagrange (λ) representa la tasa de cambio del valor óptimo de la función con respecto a cambios en la restricción, a menudo interpretado como una "fuerza de restricción". YouTube +4

Singular value decomposition
https://medium.com/@krasniuk-ai/the-math-behind-the-singular-value-decomposition-a847abf22fc1

- **Latent Spaces** - Similar "vector arithmetic" and interpretable direction results have also been found for generative adversarial networks (e.g. [13]
).
- **Latent Spaces** - Similar "vector arithmetic" and interpretable direction results have also been found for generative adversarial networks (e.g.
  - **Unsupervised representation learning with deep convolutional generative adversarial networks**
    A. Radford, L. Metz, S. Chintala.
    arXiv preprint arXiv:1511.06434. 2015.
-

[13]
).

subyacente

adjective

/subɟa'θente/

Add to word list
que está por debajo o detrás de otra cosa

¿Qué es el espacio latente?

Un espacio latente en machine learning (ML) es una representación comprimida de puntos de datos que conserva solo las características esenciales que informan la estructura subyacente de los datos de entrada. Modelar eficazmente el espacio latente es una parte integral del deep learning, incluida la mayoría de los algoritmos de IA generativa (gen AI).
El mapeo de puntos de datos en un espacio latente puede expresar datos complejos de forma eficiente y significativa, mejorando la capacidad de los modelos de machine learning para comprenderlos y manipularlos, al tiempo que se reducen los requisitos computacionales. Con ese fin, la codificación de representaciones espaciales latentes suele implicar cierto grado de reducción de la dimensionalidad: la compresión de datos de alta dimensión en un espacio de menor dimensión que omite información irrelevante o redundante.
Los espacios latentes desempeñan un papel importante en muchos campos de la ciencia de datos, y la codificación del espacio latente es un paso esencial en muchos algoritmos modernos de inteligencia artificial (IA). Por ejemplo, cualquier modelo generativo, como los autocodificadores variacionales (VAE) y las redes generativas antagónicas (GAN), calculan el espacio latente de los datos de entrenamiento para luego interpolarlos para generar nuevas muestras de datos. Los modelos de visión artificial entrenados para tareas de clasificación como la detección de objetos o la segmentación de imágenes asignan datos de entrada al espacio latente para aislar sus cualidades que son relevantes para hacer predicciones precisas.

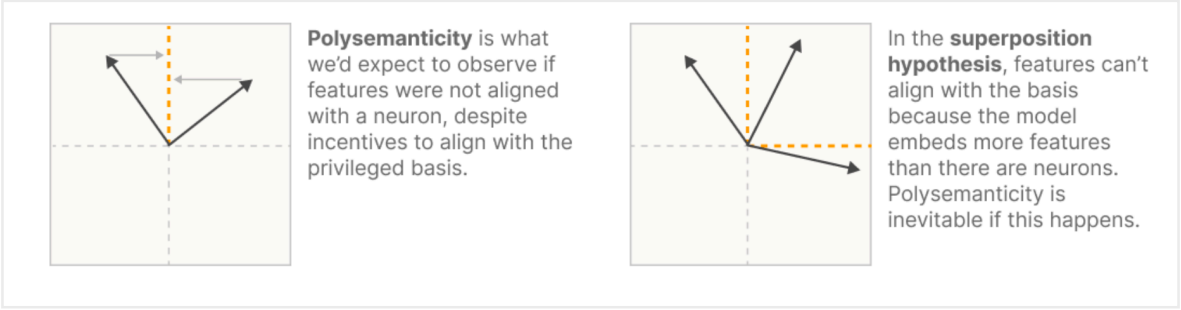
Para inspeccionar estos espacios se aplican proyecciones a 2-3 dimensiones, como t-SNE, UMAP o PCA. Técnicas como t-SNE preservan la estructura local, mientras que PCA mantiene varianza global; ninguna conserva distancias absolutas, por lo que su interpretación depende del contexto.[2

Contrario a la idea de "robar", los modelos de IA no copian y pegan imágenes

de sus conjuntos de datos de entrenamiento. En cambio, la IA descompone cada imagen en un montón de valores numéricos llamados **parámetros**: básicamente, perillas ajustables que aprenden las reglas fundamentales de lo que compone una imagen, como líneas, texturas y relaciones de color. Estos parámetros forman colectivamente un mapa comprimido y abstracto de conceptos conocido como el **espacio latente**; piénsalo no como una galería, sino como un sistema de coordenadas donde el concepto de "gatitud" está cerca de "peludito". Esta compresión es esencial, ya que un modelo de unos pocos gigabytes no puede almacenar los petabytes de datos de entrenamiento de los que aprendió, lo que demuestra que no es una base de datos de copias. En última instancia, este espacio latente contiene el potencial de todas las combinaciones posibles dentro de sus límites aprendidos, lo que permite a la IA generar una imagen nueva de un "perro azul" no recordando una foto específica, sino combinando sus conceptos aprendidos de "azul" y "perro" de diferentes puntos en su mapa interno.
Este espacio latente puede ser imaginado como un universo conceptual cuyo potencial contiene cada imagen posible que haya existido, exista actualmente o pueda existir a lo largo del tiempo, pero solo dentro de los límites estrictos de sus parámetros aprendidos, como el tamaño de la imagen y la profundidad de color. Dentro de este potencial vasto pero finito, el modelo de IA no es el universo en sí mismo, sino más bien un mapa sofisticado. El prompt de texto de un usuario actúa como un conjunto de coordenadas, guiando al modelo a un punto específico dentro de este inmenso espacio para revelar el concepto o imagen que reside allí. Esto replantea la IA no como un creador con memoria, sino como una herramienta innovadora para navegar y visualizar las ideas contenidas dentro de este paisaje de potencial finito.
Además, el acto de "robar" en un contexto artístico típicamente implica la suplantación o el engaño: un usuario humano que afirma que una pieza generada por IA es su propio trabajo original o, lo que es más grave, el trabajo de un artista humano específico. El modelo de IA en sí mismo no tiene tal intención; por lo tanto, el proceso de entrenarlo para aprender patrones no es inherentemente robo. La zona gris ética emerge cuando las corporaciones usan estos modelos para lucrar con estilos y estéticas derivados del trabajo de los artistas sin compensación, potencialmente reemplazando la necesidad de contratarlos. Esto no es robo directo, ya que no están suplantando directamente a los artistas fuente. El argumento predominante es que si el contenido está disponible públicamente para que un humano lo vea y aprenda de él, una IA debería poder hacer lo mismo. El modelo, en esencia, replica el proceso humano de inspiración: observar innumerables ejemplos para comprender un estilo, pero a una velocidad y escala sobrehumanas, lo que hace que el debate sea menos sobre el acto de aprender y más sobre la ética de su aplicación comercial.
https://www.reddit.com/r/aiwars/comments/1lv58p1/latent_space_the_simple_concept_that_prove_ai/?tl=es-419

En estadística, las **variables latentes** (o variables ocultas, en contraposición a

las variables observables), son las variables que no se observan directamente, sino que son inferidas (a través de un modelo matemático) a partir de otras variables que se observan (medidas directamente).



Referencias

Zoom In: An Introduction to Circuits

To publish :

https://distill.pub/